SVEUČILIŠTE J.J. STROSSMAYER U OSIJEKU

FAKULTET ELEKTROTEHNIKE, RAČUNARSTVA I INFORMACIJSKH TEHNOLOGIJA OSIJEK

Sveučilišni studij

UGRADBENI RAČUNALNI SUSTAVI

Profesor/mentor: **prof. dr. sc. Tomislav Keser**

TEHNIČKA DOKUMENTACIJA PROJEKTA

OPTICAL OBJECT TRACKER

Daniel Anđelić, Anto Kožarević

Akademska godina 2025/2026

Osijek, 13. 05. 2026.

Sadržaj

1. UVOD 1
1.1. Zadatak i struktura rada 2
2. TEORIJSKA PODLOGA SUSTAVA ZA OPTIČKO PRAĆENJE OBJEKTA 3
2.1. Teorijski osvrt na detekciju i praćenje objekta 3
2.2. Prijedlog sklopovskog rješenja 4
2.3. Prijedlog programskog rješenja 5
3. REALIZACIJA SUSTAVA ZA OPTIČKO PRAĆENJE OBJEKTA 6
3.1. Korištene komponente, alati i programska okruženja 6
3.2. Realizacija konstrukcijskog i sklopovskog rješenja 7
3.3. Realizacija programskog rješenja 9
4. TESTIRANJE I REZULTATI 13
4.1. Metodologija testiranja 13
4.2. Rezultati testiranja 13
5. ZAKLJUČAK 19
LITERATURA
PRILOZI I DODACI

1. UVOD

Sustavi za optičko praćenje objekata koriste se u različitim područjima tehnike, automatizacije i robotike. Njihova osnovna zadaća je prikupljanje slike iz okoline, izdvajanje traženog objekta iz slike te određivanje njegova položaja u prostoru ili ravnini slike. Dobiveni položaj objekta zatim se može koristiti za donošenje upravljačke odluke, primjerice za pomicanje kamere, robotske ruke, manipulatora ili drugog aktuatorskog sustava.

U industrijskim sustavima optičko praćenje često se koristi za nadzor proizvodnih procesa, sortiranje objekata, prepoznavanje oblika i provjeru položaja dijelova na pokretnoj traci. Slični principi koriste se i u mobilnoj robotici, gdje se kamera može koristiti za pronalazak cilja, izbjegavanje prepreka ili orijentaciju robota u prostoru. U jednostavnijim edukacijskim izvedbama optičko praćenje najčešće se temelji na *web* kameri ili kameri mikroupravljačkog sustava, algoritmu za obradu slike i servo motorima koji fizički usmjeravaju kameru ili manipulator prema detektiranom objektu.

Postojeća rješenja mogu se podijeliti u nekoliko skupina. Prvu skupinu čine sustavi koji prate objekt prema boji. Takvi sustavi izdvajaju određeni raspon boja u slici i prema njemu određuju položaj objekta. Drugu skupinu čine sustavi koji prate objekt prema obliku, pri čemu se analiziraju rubovi, konture i geometrijska svojstva objekta. Treću skupinu čine napredniji sustavi koji koriste metode strojnog učenja i neuronske mreže za prepoznavanje objekata u složenijim uvjetima. U ovom projektu primijenjen je pristup temeljen na računalnom vidu i detekciji oblika pomoću kontura.

Motiv za odabir ovakvog rješenja je izrada funkcionalnog i preglednog sustava koji pomoću kamere prepoznaje zadani objekt, određuje njegove koordinate u slici i zatim usmjerava pomični mehanizam prema njemu. Time se prikazuje osnovni princip rada robotskog optičkog sustava, pri čemu se povezuju obrada slike, serijska komunikacija i upravljanje servo motorima.

1.1. Zadatak i struktura rada

Zadatak projekta je izraditi sustav za optičku detekciju objekta i usmjeravanje pomičnog mehanizma prema detektiranom objektu. Sustav treba omogućiti odabir vrste objekta koji se prati putem jednostavnog korisničkog sučelja. U izrađenom programskom rješenju moguće je odabrati praćenje kruga, kvadrata ili trokuta. Kamera šalje video tok računalu, računalni program obrađuje sliku i određuje položaj odabranog oblika, a zatim se koordinate objekta šalju mikroupravljaču ESP32 [4]. Mikroupravljač na temelju primljenih koordinata upravlja servo motorima koji pomiču mehanizam po dvije osi.

Dokumentacija je organizirana u pet glavnih poglavlja. U prvom poglavlju dan je uvod u problem optičkog praćenja objekata te je objašnjen zadatak projekta. U drugom poglavlju opisana je teorijska podloga rada sustava, uključujući osnovne principe obrade slike, detekcije kontura i određivanja položaja objekta. U trećem poglavlju prikazana je realizacija sustava, korištene komponente, programska okruženja i način rada programa. U četvrtom poglavlju opisana je metodologija testiranja i prikazani su rezultati testiranja. U petom poglavlju dan je zaključak s osvrtom na ostvarenu funkcionalnost, prednosti, nedostatke i moguće dorade sustava.

Sustavi za optičko praćenje objekata u realnom vremenu danas imaju široku primjenu u industrijskoj automatici, naprednim sustavima za pomoć vozaču (*Advanced Driver Assistance Systems – ADAS*) te u vojne svrhe za navođenje i ciljanje. U komercijalnom i industrijskom sektoru ističu se tri karakteristična postojeća rješenja:

1. **Sustavi za automatsko praćenje i inspekciju u industriji (npr. Cognex In-Sight sustavi):** Koriste se na proizvodnim trakama za brzu identifikaciju, lokalizaciju i praćenje objekata na temelju njihovih geometrijskih značajki ili oblika [1]. Ovi sustavi omogućuju robotskim rukama da precizno manipuliraju objektima u pokretu.
2. **Sustavi za praćenje unutar autonomnih vozila (npr. Mobileye EyeQ):** Specijalizirani ugradbeni računalni sustavi koji u realnom vremenu analiziraju video tok s kamera vozila, vrše detekciju i praćenje prometnih traka, prometnih znakova, pješaka i drugih vozila radi autonomnog upravljanja i kočenja [2].
3. **PTZ (*Pan-Tilt-Zoom*) nadzorne kamere s pametnim praćenjem (npr. Hikvision Smart PTZ):** Koriste motorizirane mehanizme upravljane algoritmima računalnog vida kako bi samostalno rotirale kameru po horizontalnoj i vertikalnoj osi, zadržavajući detektiranu osobu ili vozilo u samom središtu kadra [3].

2. TEORIJSKA PODLOGA SUSTAVA ZA OPTIČKO PRAĆENJE OBJEKTA

U ovom poglavlju opisuje se teorijska podloga sustava za optičku detekciju i praćenje objekta. Objašnjava se matematički i algoritamski način digitalizacije i dobivanja slike, metode za osnovnu obradu i filtriranje slikovnih podataka, postupci za izdvajanje rubova, detekcija kontura te konačno prepoznavanje zadanih geometrijskih oblika pomoću računalnog vida (*computer vision*). U nastavku poglavlja definira se i formalno analizira idejno sklopovsko te programsko rješenje sustava prije same fizikalne realizacije.

2.1. Teorijski osvrt na detekciju i praćenje objekta

Optičko praćenje objekta temelji se na analizi digitalne slike. Digitalna slika sastoji se od piksela, pri čemu svaki piksel sadrži informaciju o intenzitetu svjetlosti i boji. U ovom projektu slika se najprije dohvaća s kamere u obliku video toka, a zatim se svaki okvir slike obrađuje zasebno.

Prvi korak obrade slike je pretvorba slike u nijanse sive. Time se smanjuje količina podataka jer se boja ne koristi kao glavni kriterij detekcije, nego se naglasak stavlja na oblik objekta. Nakon toga primjenjuje se zamućenje slike kako bi se smanjio utjecaj šuma i sitnih nepravilnosti. Zamućenje omogućuje stabilniju detekciju rubova jer se uklanjaju manje promjene intenziteta koje nisu važne za prepoznavanje oblika.

Nakon filtriranja slike provodi se detekcija rubova. Rubovi predstavljaju dijelove slike na kojima dolazi do nagle promjene intenziteta. U ovom projektu koristi se Cannyjeva metoda detekcije rubova. Nakon pronalaska rubova izdvajaju se konture, odnosno zatvorene ili djelomično zatvorene linije koje opisuju granice objekata u slici.

Za svaku pronađenu konturu izračunavaju se osnovna geometrijska svojstva. Najvažniji parametri su površina konture, opseg konture, broj vrhova aproksimirane konture i kružnost. Ako je površina konture premala, kontura se odbacuje jer se smatra da ne predstavlja traženi objekt. Broj vrhova koristi se za prepoznavanje trokuta i kvadrata, dok se kružnost koristi za prepoznavanje kruga. Na taj način moguće je razlikovati osnovne geometrijske oblike.

Kada se pronađe objekt koji odgovara odabranoj vrsti oblika, određuje se njegov položaj u slici. Položaj se dobiva iz pravokutnika koji obuhvaća konturu. Središte objekta računa se kao sredina širine i visine tog pravokutnika. Dobivene koordinate predstavljaju položaj objekta u koordinatnom sustavu slike, gdje je širina slike 640 piksela, a visina slike 480 piksela.

Kako bi se smanjilo trzanje pomičnog mehanizma, primjenjuje se zaglađivanje koordinata. Umjesto da se aktuatorima šalje svaka nova izmjerena vrijednost bez obrade, nova koordinata kombinira se s prethodnom koordinatom. Time se dobiva stabilnije kretanje i smanjuju nagle promjene položaja.

2.2. Prijedlog sklopovskog rješenja

Predloženo sklopovsko rješenje sastoji se od kamere, računala, mikroupravljača ESP32, aktuatora za pomicanje po dvije osi i konstrukcije koja nosi kameru ili objekt praćenja. Kamera prikuplja video zapis i šalje ga računalu putem mrežnog video toka. Računalo obrađuje sliku i određuje položaj objekta. Nakon obrade, računalo putem serijske komunikacije šalje koordinate mikroupravljaču ESP32.

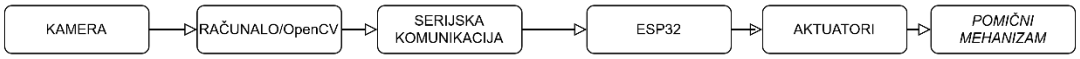
Mikroupravljač ESP32 prima koordinate objekta u obliku tekstualne poruke. Poruka sadrži koordinatu x i koordinatu y, odvojene zarezom. Nakon primitka podataka mikroupravljač provjerava jesu li koordinate unutar očekivanog raspona. Ako su podaci ispravni, koordinate se preslikavaju u kutove zakreta aktuatora.

Sustav ima dvije osi gibanja. Prva os omogućuje pomicanje u horizontalnom smjeru, a druga os omogućuje pomicanje u vertikalnom smjeru. Na taj se način može ostvariti usmjeravanje mehanizma prema detektiranom objektu u ravnini slike. U izvedbi prikazanoj programskim kodom aktuatori su izvedeni kao dva servo motora spojena na priključke GPIO 18 i GPIO 19 mikroupravljača ESP32.

Funkcionalni blokovi sustava su:

- kamera za dohvat slike,
- računalo za obradu slike,
- *web* sučelje za odabir objekta,
- serijska komunikacija između računala i ESP32 mikroupravljača,
- ESP32 mikroupravljač za upravljanje aktuatorima,
- aktuatori za pomicanje mehanizma po dvije osi,
- konstrukcija koja povezuje sve funkcionalne dijelove u cjelinu.

Slika 2.1. Blokovski prikaz predloženog sklopovskog rješenja.



Slika 2.1. *Blokovski prikaz sklopovskog rješenja sustava za optičko praćenje objekta.*

2.3. Prijedlog programskog rješenja

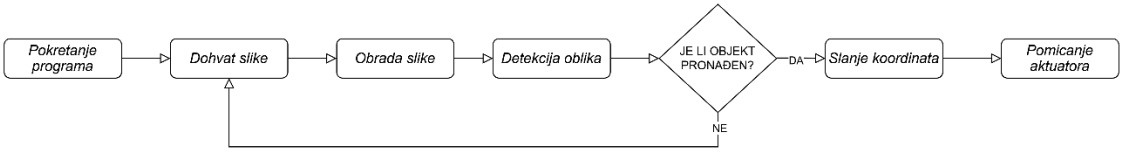
Programsko rješenje sastoji se od dva glavna dijela. Prvi dio izvodi se na računalu i zadužen je za dohvat slike, obradu slike, detekciju objekta, *web* sučelje i slanje koordinata. Drugi dio izvodi se na ESP32 mikroupravljaču i zadužen je za prihvat koordinata te upravljanje aktuatorima.

Računalni program dohvaća video tok s kamere, mijenja veličinu slike na 640 x 480 piksela i pretvara sliku u nijanse sive. Nakon toga provodi se zamućenje slike i detekcija rubova. Iz dobivenih rubova izdvajaju se konture, a svaka kontura analizira se funkcijom za prepoznavanje oblika. Ako oblik odgovara trenutno odabranom cilju, sprema se kontura s najvećom površinom. Za odabranu konturu računa se središte, koordinate se zaglađuju i šalju na ESP32.

Web sučelje omogućuje korisniku odabir oblika koji se prati. Korisnik može odabrati krug, kvadrat ili trokut. Sučelje prikazuje trenutno odabrani objekt, status detekcije i video prikaz s označenim objektom. Ako je objekt detektiran, na slici se prikazuju okvir oko objekta, središnja točka i tekstualna informacija o koordinatama. Ako objekt nije detektiran, prikazuje se poruka da traženi objekt nije pronađen.

Program na ESP32 mikroupravljaču stalno provjerava postoje li novi podaci na serijskom ulazu. Kada primi podatke, traži zarez koji razdvaja koordinatu x i koordinatu y. Nakon pretvorbe teksta u cijele brojeve provjerava se jesu li koordinate unutar dopuštenog raspona. Ako su koordinate ispravne, vrijednosti se preslikavaju u kutove zakreta aktuatora i šalju servo motorima.

Slika 2.2. Blok dijagram predloženog programskog rješenja.



Slika 2.2. *Blok dijagram programskog rješenja sustava za praćenje objekta.*

3. REALIZACIJA SUSTAVA ZA OPTIČKO PRAĆENJE OBJEKTA

U ovom poglavlju detaljno se opisuje stvarna fizikalna i programska realizacija razvijenog sustava za praćenje. Prikazane su sve korištene sklopovske komponente, elektronički moduli, razvojni alati i programska okruženja te je detaljno dokumentiran način međusobnog povezivanja pojedinih dijelova sustava kroz električne sheme. Poseban naglasak stavljen je na implementaciju programskog rješenja na strani osobnog računala unutar okruženja *Python* i knjižnice *OpenCV*, kao i na ugradbenom programu (*firmware*) unutar mikroupravljača *ESP32*.

3.1. Korištene komponente, alati i programska okruženja

Za izradu projekta korištene su sljedeće komponente:

- **ESP32-CAM modul**
  ESP32-CAM modul koristi se za dohvat slike i slanje video toka prema računalu. Kamera omogućuje praćenje scene ispred sustava i predstavlja ulazni senzor cijelog sustava.

- **ESP32 DevKit mikroupravljačka pločica**
  ESP32 DevKit koristi se za prihvat koordinata objekta putem serijske komunikacije i upravljanje aktuatorima. U projektu se koristi kao upravljačka jedinica koja povezuje računalnu obradu slike s fizičkim pomicanjem mehanizma.

- **Dva aktuatora za dvije osi gibanja**
  U programskom kodu korišteni su servo motori. Jedan aktuator služi za pomicanje po horizontalnoj osi, a drugi za pomicanje po vertikalnoj osi. Na taj način omogućuje se usmjeravanje mehanizma prema detektiranom objektu.

- **Eksperimentalna pločica i spojni vodiči**
  Eksperimentalna pločica i vodiči korišteni su za povezivanje modula i olakšano ispitivanje sustava tijekom razvoja.

- **Konstrukcija pomičnog mehanizma**
  Konstrukcija omogućuje fizičko postavljanje kamere i aktuatora. U prikazanoj izvedbi korištena je jednostavna konstrukcija sastavljena od dostupnih elemenata, pri čemu je cilj bio ostvariti funkcionalnu provjeru rada sustava.

Korišteni programski alati i programska okruženja:

- **Python**
  Python se koristi za izradu glavnog računalnog programa za obradu slike, *web* sučelje i serijsku komunikaciju.

- **OpenCV**
  OpenCV se koristi za obradu slike, detekciju rubova, pronalazak kontura i označavanje detektiranog objekta.

- **Flask**
  Flask se koristi za izradu jednostavnog *web* sučelja putem kojeg korisnik odabire oblik koji se prati i promatra video prikaz.

- **Arduino IDE**
  Arduino IDE koristi se za pisanje i prijenos programa na ESP32 DevKit pločicu.

- **ESP32Servo biblioteka**
  Biblioteka ESP32Servo [5] koristi se za upravljanje servo motorima pomoću ESP32 mikroupravljača.

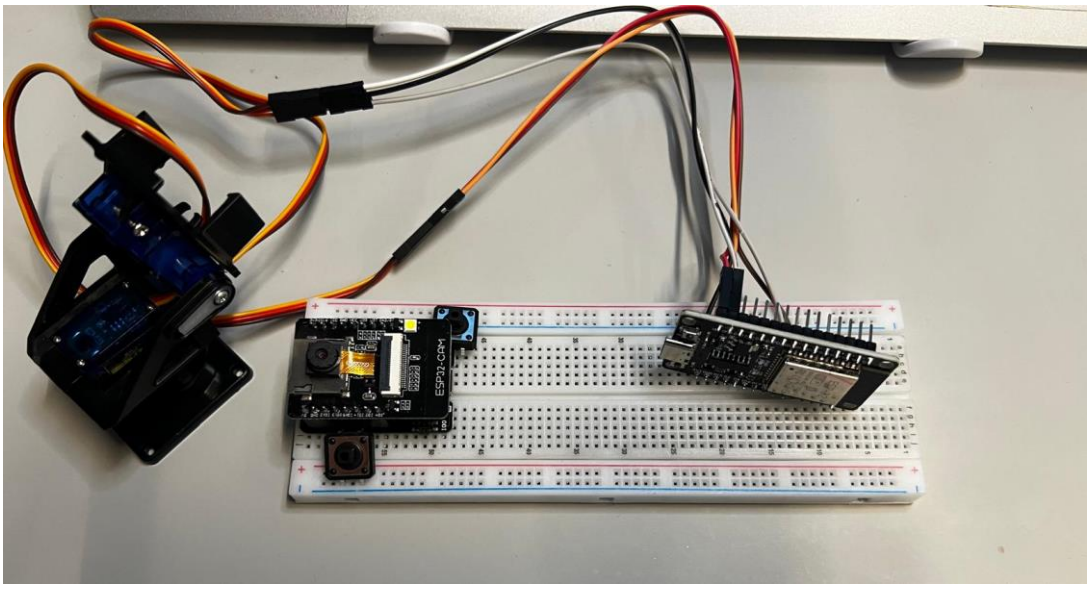
3.2. Realizacija konstrukcijskog i sklopovskog rješenja

Sustav je realiziran povezivanjem kamere, računala, *ESP32 DevKit* pločice i aktuatora. Kamera je postavljena tako da snima područje u kojem se nalazi objekt koji se prati. Video tok s kamere dohvaća se u *Python* programu preko mrežne adrese. Računalo obrađuje sliku i šalje koordinate objekta *ESP32 DevKit* pločici putem serijskog priključka.

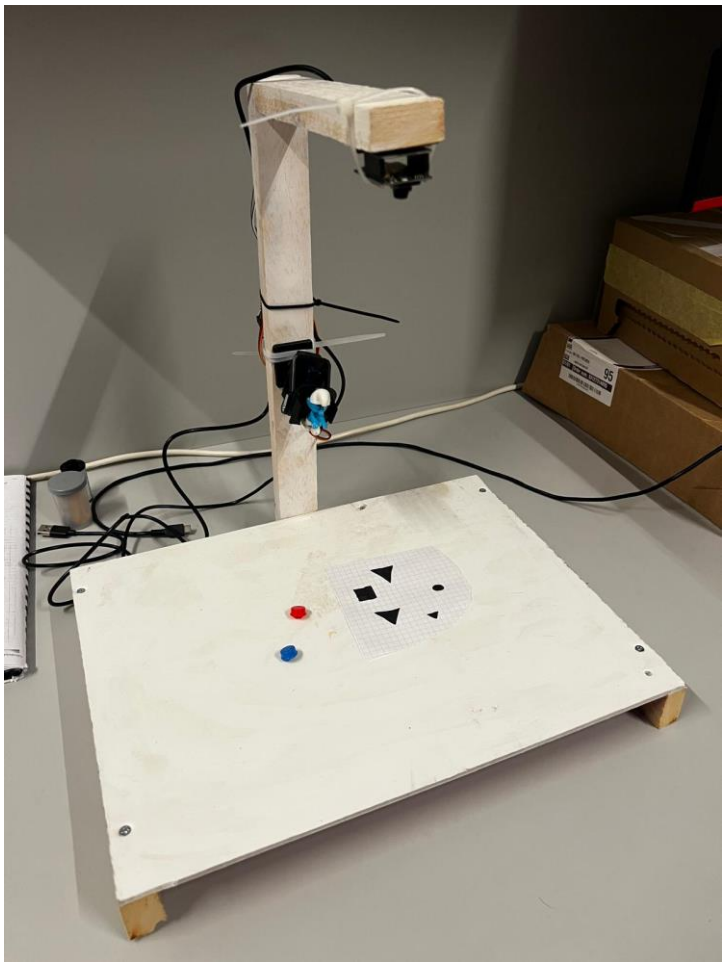
ESP32 DevKit pločica spojena je s aktuatorima koji omogućuju gibanje po dvije osi. U programu su definirani priključci GPIO 18 i GPIO 19. Prvi priključak koristi se za upravljanje aktuatorom horizontalne osi, a drugi priključak za upravljanje aktuatorom vertikalne osi. Početni položaj oba aktuatora postavljen je na 90°, što predstavlja srednji položaj.

Na temelju primljenih koordinata računa se kut zakreta. Koordinata x preslikava se iz raspona od 0 do 640 piksela u raspon kutova od 150° do 30°. Time se ostvaruje suprotan smjer pomaka za horizontalnu os. Koordinata y preslikava se iz raspona od 0 do 480 piksela u raspon kutova od 30° do 150°. Nakon preslikavanja, kutovi se dodatno ograničavaju na raspon od 30° do 150° kako bi se spriječilo pomicanje aktuatora izvan sigurnog područja rada.

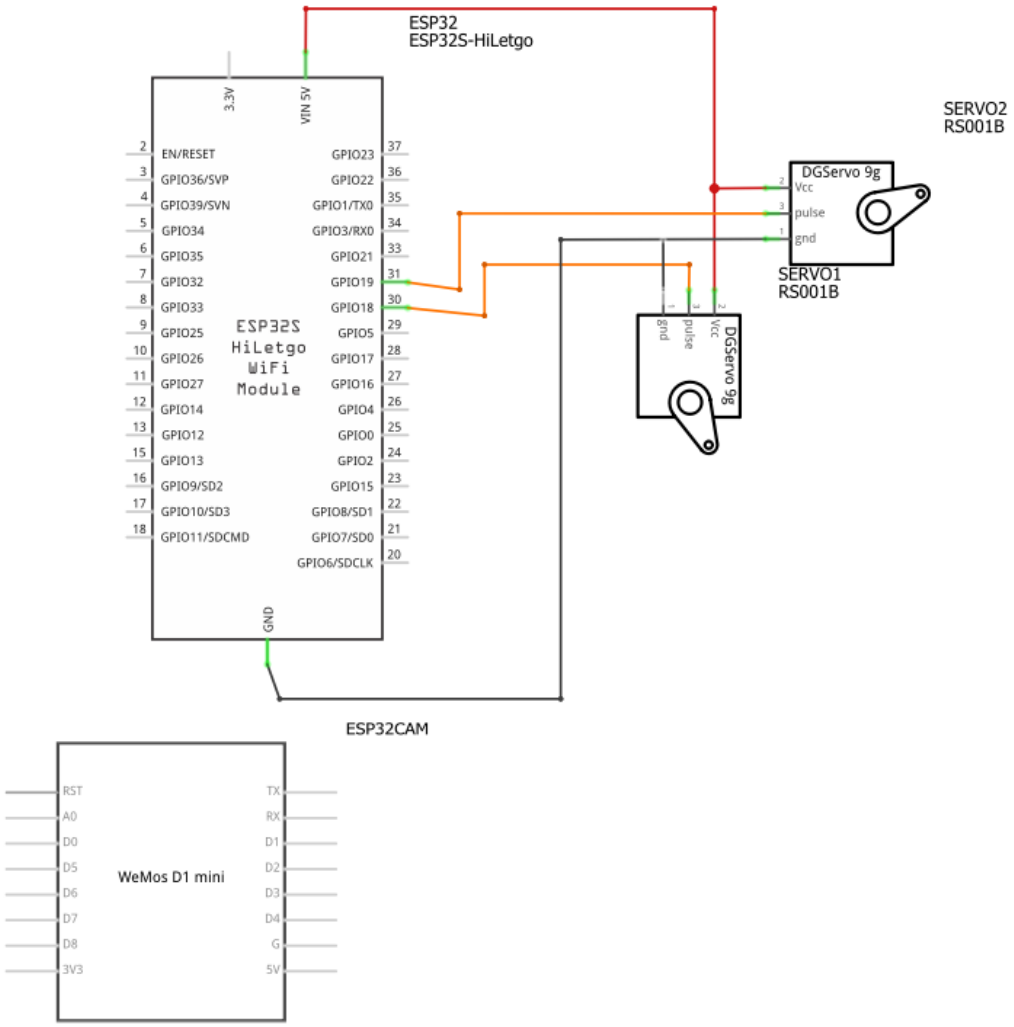
Na fotografijama projekta vidljivo je da je kamera postavljena na eksperimentalnu pločicu, dok je *ESP32 DevKit* pločica spojena vodičima s aktuatorima. Konstrukcija je izvedena kao jednostavan prototip koji omogućuje provjeru rada algoritma za praćenje objekta i upravljanja mehanizmom.



Slika 3.1. *Prikaz korištenih elektroničkih komponenti sustava.*



Slika 3.2. *Prikaz konstrukcije sustava tijekom rada.*



Slika 3.3. *Shema povezivanja ESP32-CAM modula, ESP32 DevKit pločice i servo motora.*

3.3. Realizacija programskog rješenja

Programsko rješenje realizirano je pomoću dva programa. Prvi program napisan je u *Pythonu* i izvodi se na računalu. Drugi program napisan je za *ESP32 DevKit* i izvodi se na mikroupravljaču.

Python program započinje uvozom potrebnih biblioteka. Biblioteka *OpenCV* koristi se za obradu slike, *NumPy* za matematičke operacije, *serial* za serijsku komunikaciju, time za vremenska kašnjenja i *Flask* za *web* sučelje. Nakon pokretanja programa otvara se serijska komunikacija prema *ESP32 DevKit* pločici na brzini od 115200 bit/s. Također se otvara video tok s *ESP32-CAM* modula.

U programu je definirana varijabla TARGET_SHAPE, koja predstavlja oblik koji sustav trenutno prati. Početna vrijednost je CIRCLE, odnosno krug. Korisnik putem *web* sučelja može promijeniti oblik koji se prati na SQUARE ili TRIANGLE.

Funkcija za detekciju oblika prima konturu i analizira njezina geometrijska svojstva. Ako je površina konture manja od zadane granice, kontura se odbacuje. Zatim se računa opseg konture i aproksimacija konture. Ako aproksimirana kontura ima tri vrha, prepoznaje se kao trokut. Ako ima četiri vrha i omjer širine i visine odgovara približno pravilnom obliku, prepoznaje se kao kvadrat. Ako kontura ima dovoljnu kružnost, prepoznaje se kao krug.

Glavna funkcija za generiranje video okvira neprekidno dohvaća sliku s kamere. Svaki okvir se smanjuje na 640 x 480 piksela, pretvara u nijanse sive, zamućuje i obrađuje Cannyjevim detektorom rubova. Nakon toga pronalaze se konture. Program prolazi kroz sve konture i traži onu koja odgovara trenutno odabranom obliku. Ako postoji više odgovarajućih kontura, odabire se ona s najvećom površinom.

Kada je objekt pronađen, računa se njegovo središte. Koordinate se zaglađuju pomoću prethodne vrijednosti kako bi se smanjile nagle promjene položaja. Zaglađivanje je izvedeno tako da prethodna vrijednost ima veći utjecaj od nove vrijednosti. Nakon toga se koordinate šalju ESP32 mikroupravljaču u obliku tekstualnog zapisa x,y. Slanje se ograničava na približno svakih 0,1 s kako se serijska komunikacija ne bi nepotrebno opterećivala.

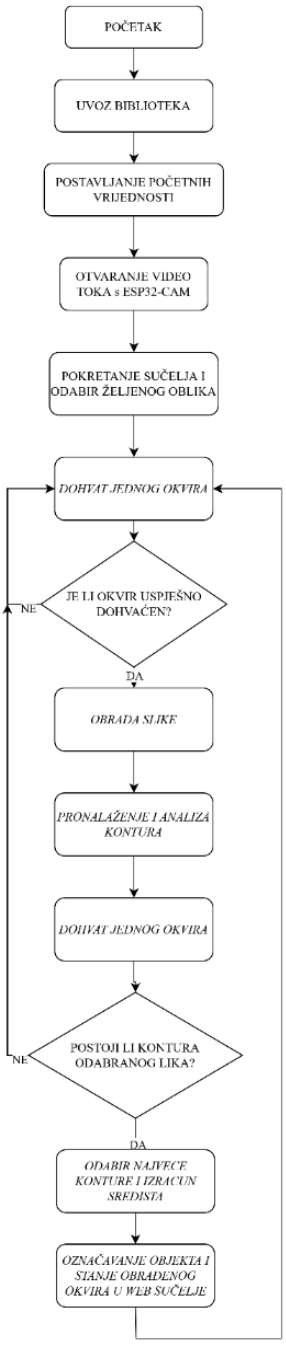
Ako objekt nije pronađen, koordinate se poništavaju i u *web* sučelju se prikazuje status da objekt nije detektiran. Na video prikazu se ispisuje poruka da traženi oblik nije pronađen.

Flask web sučelje sastoji se od početne stranice, rute za promjenu oblika i rute za video prikaz. Početna stranica prikazuje naslov projekta, trenutno odabrani oblik, status detekcije i tri gumba za odabir oblika. Ruta /set/<shape> mijenja odabrani oblik ako je unesena vrijednost jedna od dopuštenih vrijednosti. Ruta /video_feed prikazuje obrađeni video tok.

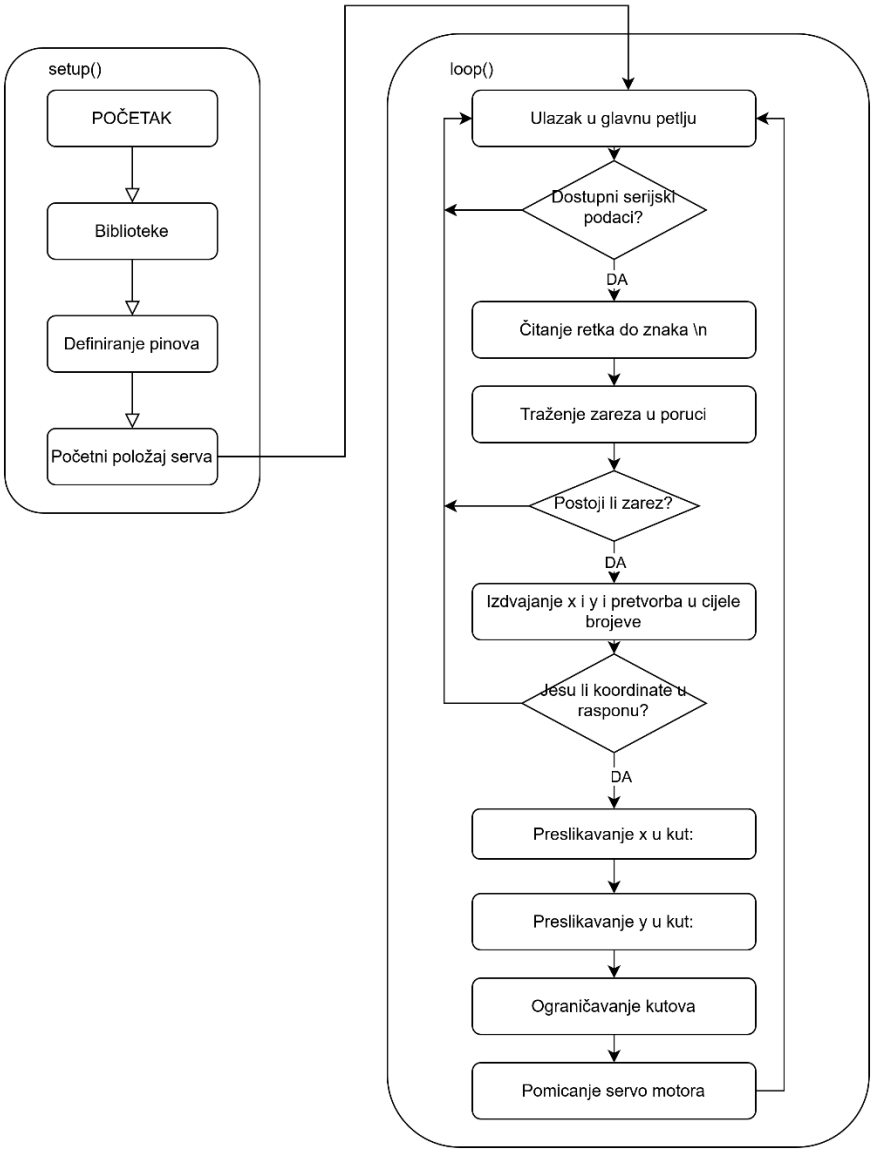
ESP32 program započinje uključivanjem biblioteke ESP32Servo i definiranjem dvaju servo objekata. U funkciji setup() pokreće se serijska komunikacija brzinom 115200 bit/s, servo motori se povezuju s priključcima GPIO 18 i GPIO 19 te se postavljaju u početni položaj od 90°. Nakon toga se preko serijskog monitora ispisuje poruka da je sustav spreman.

U funkciji loop() mikroupravljač provjerava jesu li dostupni novi serijski podaci. Ako su podaci dostupni, čita se cijeli redak do znaka za novi red. Program zatim pronalazi zarez koji razdvaja koordinatu x i koordinatu y. Koordinate se pretvaraju u cijele brojeve i provjerava se jesu li u

rasponu slike. Ako su ispravne, funkcijom map() preslikavaju se u kutove aktuatora. Nakon ograničenja kutova funkcijom constrain(), vrijednosti se šalju servo motorima.



Slika 3.5. *Detaljni dijagram toka rada Python programa za obradu slike i praćenje objekta.*



Slika 3.6. *Detaljni dijagram toka rada ESP32 programa za upravljanje servo motorima.*

4. TESTIRANJE I REZULTATI

U ovom poglavlju opisuje se način testiranja izrađenog sustava. Testiranje se provodi s ciljem provjere detekcije objekta, stabilnosti praćenja i odziva pomičnog mehanizma. Rezultati testiranja trebaju pokazati koliko pouzdano sustav prepoznaje zadani oblik i koliko uspješno usmjerava mehanizam prema objektu. Prema naputku, testiranja su opisana metodološki, a rezultati su prikazani kvantitativno, uz provođenje matrice od 20 uzastopnih pokusa radi utvrđivanja funkcionalne točnosti.

4.1. Metodologija testiranja

Testiranje sustava provedeno je kroz pet strukturiranih funkcionalnih provjera:

1. **Provjera dohvata video toka:** Kamera se postavlja ispred objekta, a zatim se u mrežnom sučelju promatra kontinuitet prijenosa video toka. Ako je video prikaz stabilan, komunikacija između kamere i računala smatra se ispravnom.

2. **Verifikacija detekcije oblika:** U kadar se sukcesivno postavljaju objekti u obliku kruga, kvadrata i trokuta. U sučelju se odabire odgovarajući oblik i promatra generira li program računalnog vida zeleni granični okvir oko objekta i njegovu središnju točku.

3. **Točnost određivanja koordinata (Kvantitativno ispitivanje):** Objekt se postavlja na različite referentne položaje u kadru (lijevo, desno, gore, dolje, sredina). Za svaki položaj očitavaju se prikazane koordinate x i y unutar rezolucije zadanog toka. Provedeno je 20 uzastopnih pokusa pozicioniranja kako bi se utvrdilo odstupanje izračunatih koordinata u odnosu na stvarne fizičke promjene položaja objekta.

4. **Odziv aktuatora i mehanizma:** Nakon detekcije, objekt se pomiče po kadru, a promatra se odziv aktuatorskog mehanizma. Prati se ponašanje sustava pri pomicanju po horizontalnoj i vertikalnoj osi te se bilježi postojanje kašnjenja, trzaja ili eventualnog preopterećenja.

5. **Ispitivanje upravljačkog sučelja:** Provjerava se odziv aplikacije prilikom prebacivanja između ponuđenih geometrijskih oblika te ispravnost prikaza statusa trenutne detekcije.

4.2. Rezultati testiranja

Tijekom ispitivanja potvrđeno je da sustav stabilno dohvaća video tok s kamere i prikazuje ga unutar mrežnog sučelja. Sučelje omogućuje trenutačni odabir objekta koji se prati (krug, kvadrat i

trokut). Kada se u kadru nalazi objekt odabranog oblika, program uspješno označava objekt zelenim pravokutnikom i projicira njegovu središnju točku.

Uočeno je da kvaliteta i robusnost detekcije ovise o razini osvjetljenja, kontrastu objekta u odnosu na pozadinu te jasnoći rubova. Objekti s oštrim rubovima i većim kontrastom detektiraju se s najvišim stupnjem pouzdanosti. Ako je objekt premalen, djelomično zaklonjen ili pozadina sadrži visoku gustoću geometrijskog šuma, moguća je slabija detekcija ili kratkotrajno pogrešno prepoznavanje.

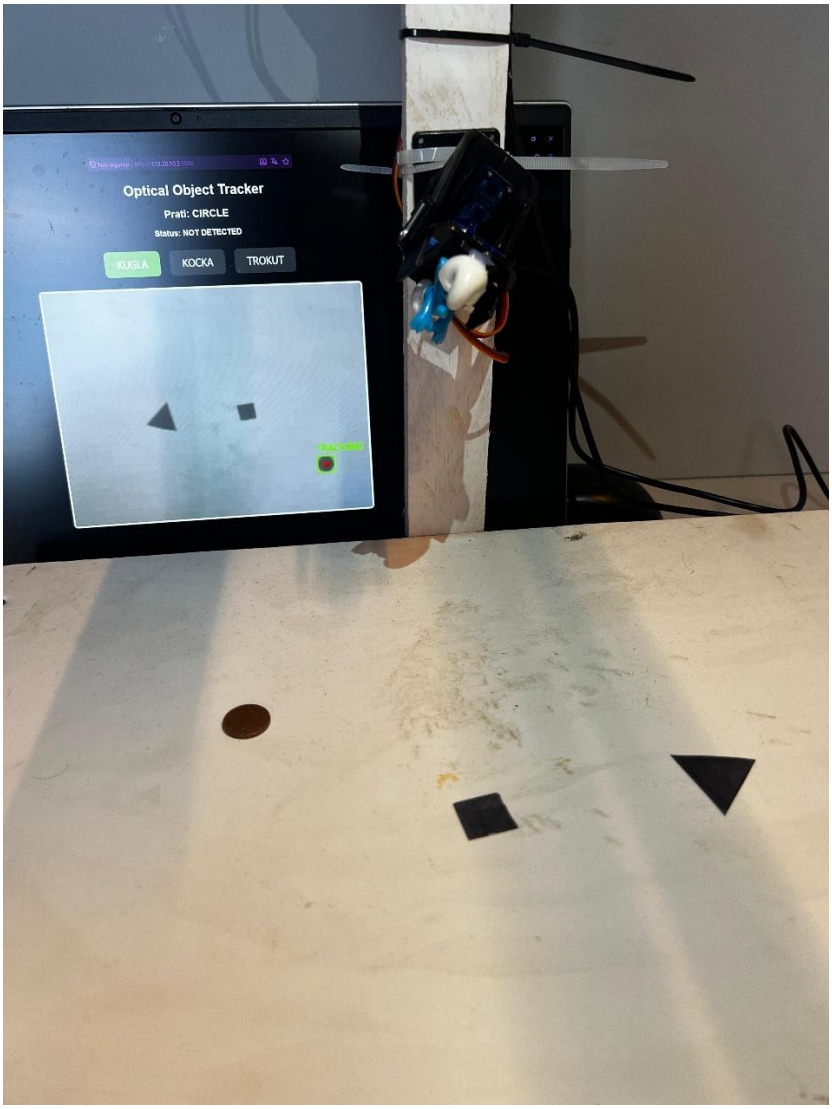
Prilikom praćenja objekta, izračunate koordinate se u realnom vremenu prosljeđuju *ESP32* mikroupravljaču koji upravlja pomicanjem aktuatorskog mehanizma. Implementirana softverska metoda za zaglađivanje koordinata uspješno eliminira nagle promjene položaja i pridonosi stabilnijem kretanju bez trzaja. Ograničavanje kutova aktuatora na raspon od 30° do 150° u potpunosti sprječava pomicanje izvan predviđenog radnog područja i štiti mehanizam od mehaničkog zaglavljivanja.

Za potrebe cjelokupne validacije i analize stabilnosti sustava kroz svih pet definiranih faza metodologije, u slici 4.1. prikazana je matrica kvantitativnih rezultata funkcionalnog ispitivanja kroz seriju od 20 uzastopnih pokusa.

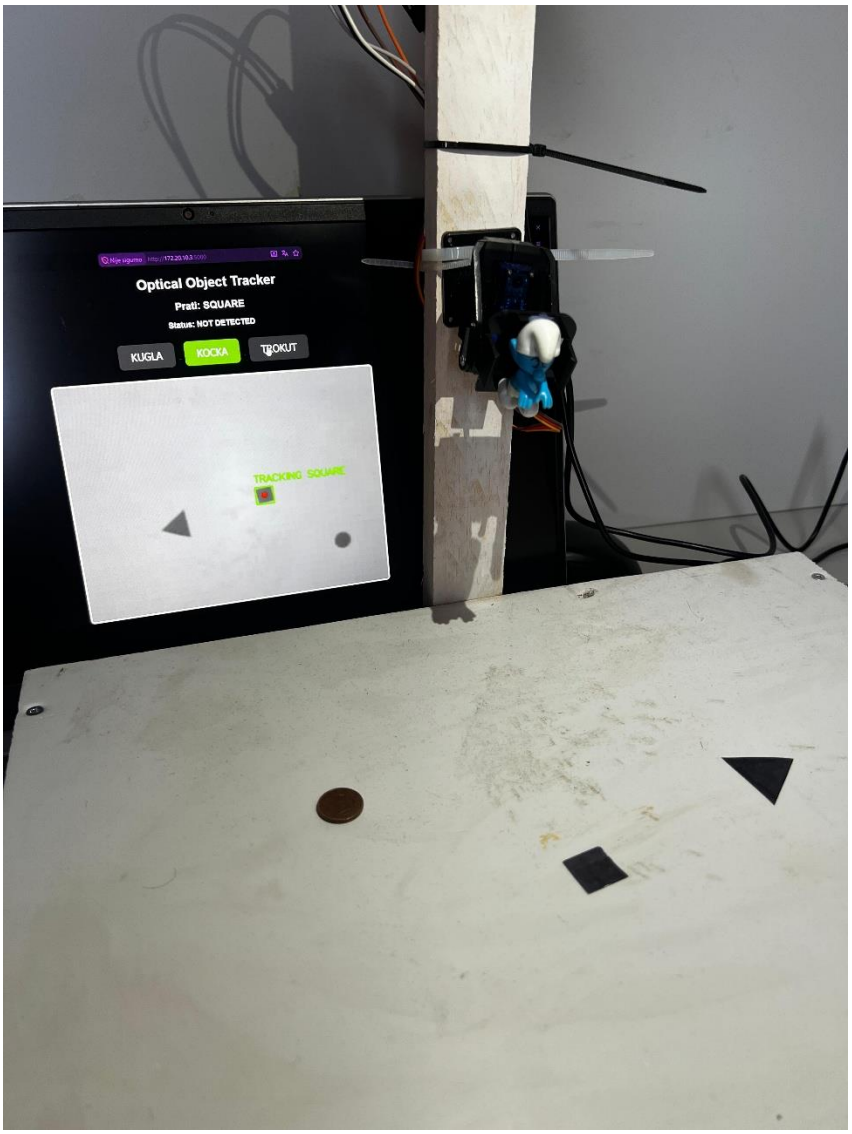
Broj pokusa (i)	P1 (Dohvat videa)	P2 (Detekcija oblika)	P3 (Točnost koordinata)	P4 (Odziv aktuatora)	P5 (Rad sučelja)
1	Uspješno	Uspješno	Uspješno	Uspješno	Uspješno
2	Uspješno	Uspješno	Uspješno	Uspješno	Uspješno
3	Uspješno	Djelomično	Uspješno	Uspješno	Uspješno
4	Uspješno	Uspješno	Uspješno	Uspješno	Uspješno
5	Uspješno	Uspješno	Djelomično	Uspješno	Uspješno
6	Uspješno	Uspješno	Uspješno	Uspješno	Uspješno
7	Uspješno	Uspješno	Uspješno	Uspješno	Uspješno
8	Neuspješno	Neuspješno	Neuspješno	Neuspješno	Uspješno
9	Uspješno	Uspješno	Uspješno	Uspješno	Uspješno
10	Uspješno	Uspješno	Djelomično	Uspješno	Uspješno
11	Uspješno	Uspješno	Uspješno	Uspješno	Uspješno
12	Uspješno	Uspješno	Djelomično	Uspješno	Uspješno
13	Uspješno	Djelomično	Uspješno	Uspješno	Uspješno
14	Uspješno	Uspješno	Djelomično	Uspješno	Uspješno
15	Uspješno	Uspješno	Uspješno	Uspješno	Uspješno
16	Uspješno	Uspješno	Uspješno	Uspješno	Uspješno
17	Uspješno	Uspješno	Djelomično	Uspješno	Uspješno
18	Uspješno	Uspješno	Uspješno	Uspješno	Uspješno
19	Uspješno	Uspješno	Djelomično	Uspješno	Uspješno
20	Uspješno	Uspješno	Uspješno	Uspješno	Uspješno

***Slika 4.1**. Matrica verifikacije i kvantitativne uspješnosti funkcionalnih parametara sustava kroz seriju od 20 uzastopnih pokusa u promjenjivim uvjetima okoline.*

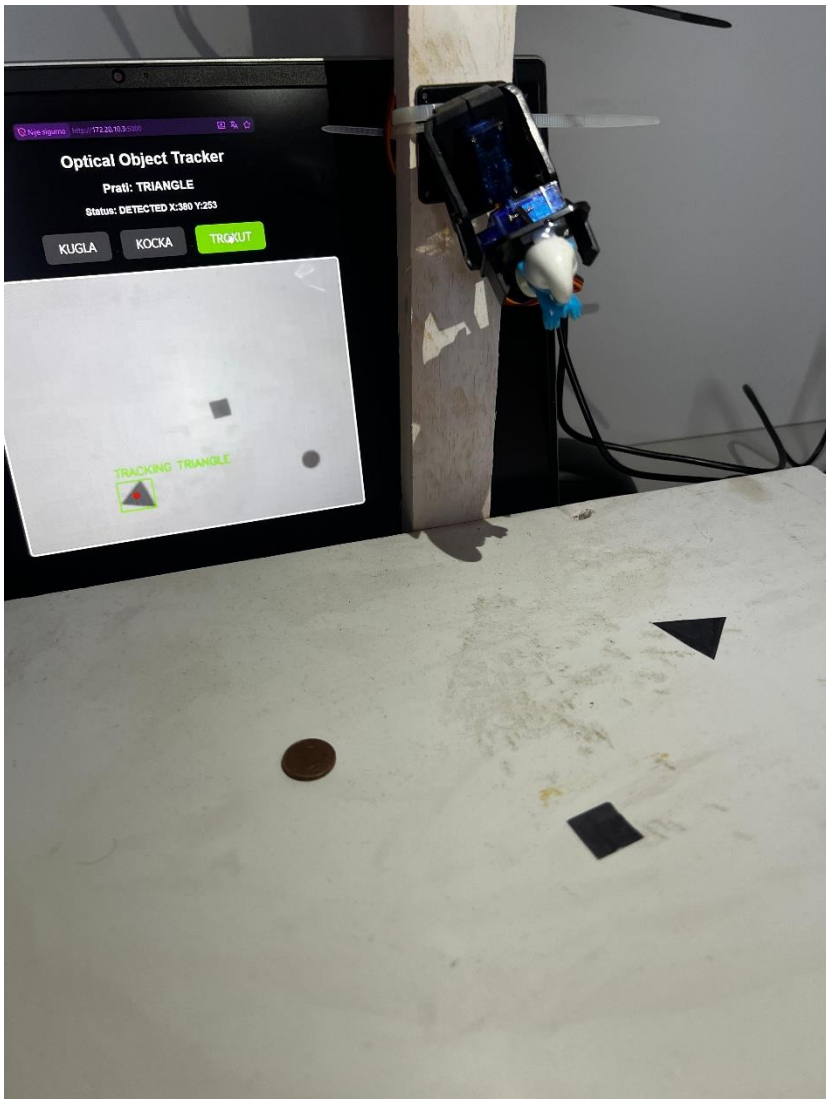
Za potrebe verifikacije robusnosti algoritma računalnog vida, na Slici 4.2., Slici 4.3. i Slici 4.4. prikazani su stvarni testni uzorci geometrijskih oblika te rezultati njihove uspješne detekcije i generiranja centroida unutar razvijenog mrežnog sučelja.



Slika 4.2. *Prikaz izvornog objekta kružnog oblika i rezultat softverske detekcije s generiranim graničnim okvirom i pripadajućim središtem.*

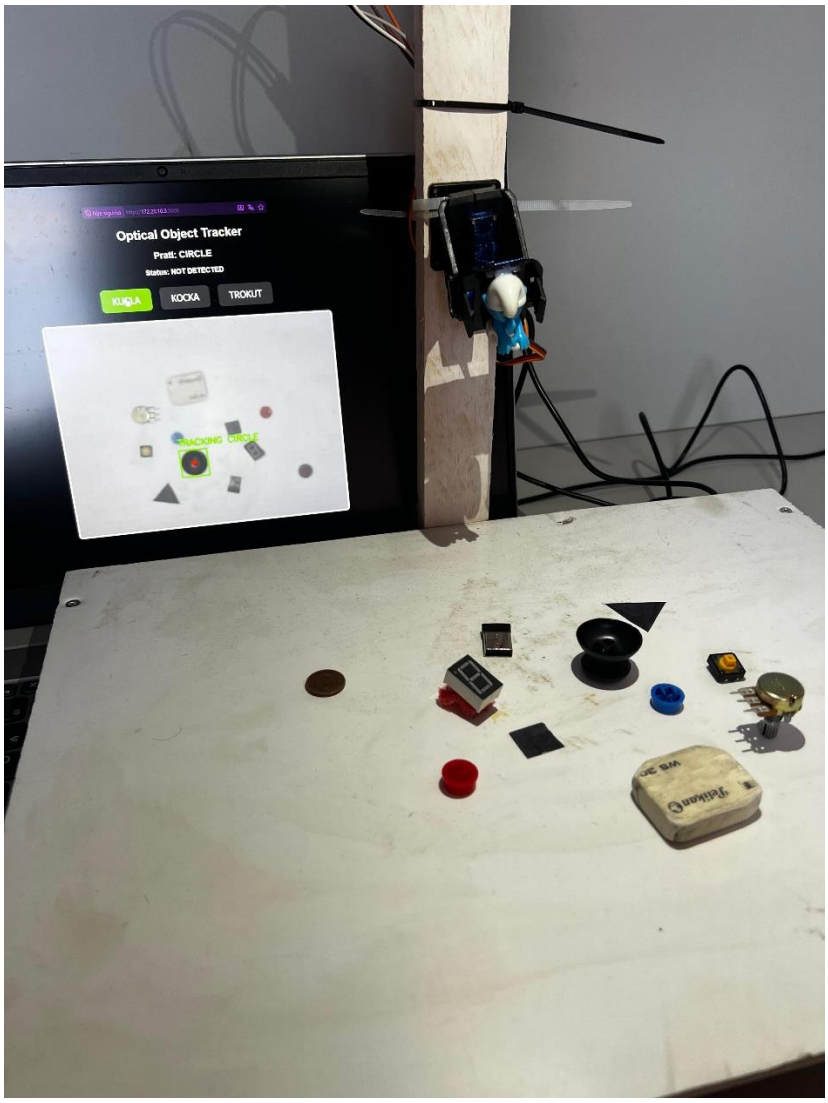


Slika 4.3. *Prikaz izvornog objekta kvadratnog oblika i rezultat softverske detekcije unutar mrežnog sučelja.*



Slika 4.4. *Prikaz izvornog objekta trokutastog oblika i rezultat softverske izolacije kontura.*

Prilikom ispitivanja ponašanja sustava u složenijim uvjetima rada, na radnu površinu postavljeno je više različitih geometrijskih oblika istovremeno. Rezultat obrade prikazan je na Slici 4.5.



Slika 4.5. *Funkcionalni prikaz obrade složene scene s više različitih geometrijskih objekata unutar kadra i selektivnim praćenjem ciljanog oblika.*

5. ZAKLJUČAK

U sklopu projekta izrađen je sustav za optičku detekciju i praćenje objekta. Sustav koristi kameru za dohvat slike, računalni program za obradu slike i mikroupravljač ESP32 za upravljanje pomičnim mehanizmom. Korisniku je omogućeno odabiranje oblika koji se prati putem jednostavnog *web* sučelja.

Realizirano programsko rješenje omogućuje detekciju kruga, kvadrata i trokuta pomoću obrade slike i analize kontura. Nakon detekcije objekta određuju se njegove koordinate, koje se šalju mikroupravljaču. ESP32 zatim koordinate pretvara u kutove zakreta servo motora. Time je ostvarena osnovna funkcionalnost optičkog praćenja objekta i usmjeravanja mehanizma prema detektiranom cilju.

Prednost izrađenog sustava je jednostavna struktura, pregledno *web* sučelje i mogućnost promjene objekta koji se prati. Sustav je pogodan za edukacijsku svrhe jer jasno prikazuje povezanost računalnog vida, serijske komunikacije i upravljanja servo motorima. Dodatna prednost je mogućnost daljnje nadogradnje, primjerice uvođenjem detekcije boje, složenijih oblika ili naprednijih metoda prepoznavanja objekata.

Nedostatci sustava odnose se na osjetljivost detekcije na osvjetljenje, pozadinu i kvalitetu rubova objekta. U složenijim uvjetima može doći do pogrešne detekcije ili gubitka objekta. Također, mehanička izvedba prototipa može uzrokovati manju stabilnost pomicanja, posebno ako servo motori ili konstrukcija nisu dovoljno čvrsto postavljeni.

Za buduće poboljšanje sustava preporučuje se izrada stabilnije konstrukcije, preciznije podešavanje algoritma detekcije, dodavanje kalibracije kamere i uvođenje naprednijeg algoritma praćenja. Također bi bilo korisno dodati automatsko centriranje objekta, pri čemu bi sustav kontinuirano uspoređivao položaj objekta sa središtem slike i prema tome upravljao servo motorima.

LITERATURA

[1] Cognex Corporation, "In-Sight Vision Systems Datasheet and Manual",

Dostupno na: https://www.cognex.com [Pristupljeno: lipanj, 2026.]

[2] Mobileye, "The Drive to Autonomous Vehicles: EyeQ System-on-Chip Technology",

White Paper, Dostupno na: https://www.mobileye.com [Pristupljeno: lipanj, 2026.]

[3] Hikvision Digital Technology, "Smart PTZ Camera Systems: Intelligent Tracking

User Guide", Tehnološki priručnik, Dostupno na: https://www.hikvision.com [Pristupljeno: lipanj, 2026.]

[4] Espressif Systems, ESP32 Technical Reference Manual, poveznica https://www.lcdwiki.com/res/E32R35T/esp32_technical_reference_manual_en.pdf [Pristupljeno: lipanj, 2026.]

[5] ESP32Servo Library Documentation, https://github.com/madhephaestus/ESP32Servo, [Pristupljeno: lipanj, 2026.]

PRILOZI I DODACI

P.1 – Python programski kod za obradu slike i web sučelje

U ovom prilogu navodi se izvorni programski kod računalnog dijela sustava. Kod obuhvaća dohvat video toka, obradu slike pomoću biblioteke *OpenCV*, detekciju oblika, *web* sučelje izrađeno pomoću *Flask* okvira i slanje koordinata prema ESP32 mikroupravljaču putem serijske komunikacije.

```
import cv2
import numpy as np
import serial
import time
import requests
from flask import Flask, Response, render_template_string, redirect

stream_url = "http://172.20.10.3:81/stream"
camera_ip = "172.20.10.3"

esp = serial.Serial("COM6", 115200, timeout=1)
time.sleep(2)

try:
    requests.get(
        f"http://{camera_ip}/control?var=led_intensity&val=120"
    )
    print("LED uključen")
except:
    print("LED nije moguće upaliti")

cap = cv2.VideoCapture(stream_url)

app = Flask(__name__)

TARGET_SHAPE = "CIRCLE"

smooth_x = None
smooth_y = None

last_send = 0
last_status = "NOT DETECTED"

html_page = """
<!DOCTYPE html>
<html>
<head>
<title>Optical Object Tracker</title>

<style>

body{
font-family:Arial;
text-align:center;
background:#111;
color:white;
}
```

```
button{
font-size:25px;
padding:15px 30px;
margin:10px;
border-radius:10px;
border:none;
cursor:pointer;
}

.active{
background:limegreen;
color:white;
}

.normal{
background:#444;
color:white;
}

img{
border:4px solid white;
border-radius:10px;
width:80%;
max-width:800px;
}

</style>

</head>

<body>

<h1>Optical Object Tracker</h1>

<h2>Prati: {{ target }}</h2>

<h3>Status: {{ status }}</h3>

<a href="/set/CIRCLE">
<button class="{{ 'active' if target=='CIRCLE' else 'normal' }}">
KUGLA
</button>
</a>

<a href="/set/SQUARE">
<button class="{{ 'active' if target=='SQUARE' else 'normal' }}">
KOCKA
</button>
</a>

<a href="/set/TRIANGLE">
<button class="{{ 'active' if target=='TRIANGLE' else 'normal' }}">
TROKUT
</button>
</a>

<br><br>
```

```
<img src="/video_feed">

</body>
</html>
"""

def detect_shape(contour):

    area=cv2.contourArea(contour)

    if area<600:
        return "UNKNOWN"

    perimeter=cv2.arcLength(contour,True)

    if perimeter==0:
        return "UNKNOWN"

    approx=cv2.approxPolyDP(
        contour,
        0.04*perimeter,
        True
    )

    vertices=len(approx)

    if vertices==3:
        return "TRIANGLE"

    if vertices==4:

        x,y,w,h=cv2.biiRect(approx)

        ratio=w/float(h)

        if 0.65<=ratio<=1.35:
            return "SQUARE"

    circularity=4*np.pi*area/(perimeter*perimeter)

    if circularity>0.55:
        return "CIRCLE"

    return "UNKNOWN"

def generate_frames():

    global smooth_x
    global smooth_y
    global last_send
    global last_status

    while True:

        ret,frame=cap.read()

        if not ret:
            continue
```

```
frame=cv2.resize(frame,(640,480))

gray=cv2.cvtColor(
    frame,
    cv2.COLOR_BGR2GRAY
)

blur=cv2.GaussianBlur(
    gray,
    (7,7),
    0
)

edges=cv2.Canny(
    blur,
    20,
    80
)

contours,_=cv2.findContours(
    edges,
    cv2.RETR_EXTERNAL,
    cv2.CHAIN_APPROX_SIMPLE
)

best_contour=None
best_area=0

for contour in contours:

    shape=detect_shape(contour)

    if shape!=TARGET_SHAPE:
        continue

    area=cv2.contourArea(contour)

    if area>best_area:
        best_area=area
        best_contour=contour

if best_contour is not None:

    x,y,w,h=cv2.boundingRect(
        best_contour
    )

    current_x=x+w//2
    current_y=y+h//2

    if smooth_x is None:
        smooth_x=current_x
        smooth_y=current_y
    else:
        smooth_x=int(
            0.8*smooth_x+0.2*current_x
        )

        smooth_y=int(
```

```
                    0.8*smooth_y+0.2*current_y
                )

            if time.time()-last_send>0.1:

                esp.write(
                    f"{smooth_x},{smooth_y}\n".encode()
                )

                last_send=time.time()

            last_status=f"DETECTED X:{smooth_x} Y:{smooth_y}"

            cv2.rectangle(
                frame,
                (x,y),
                (x+w,y+h),
                (0,255,0),
                2
            )

            cv2.putText(
                frame,
                f"TRACKING {TARGET_SHAPE}",
                (x,y-10),
                cv2.FONT_HERSHEY_SIMPLEX,
                0.7,
                (0,255,0),
                2
            )

            cv2.circle(
                frame,
                (smooth_x,smooth_y),
                6,
                (0,0,255),
                -1
            )

        else:

            smooth_x=None
            smooth_y=None
            last_status="NOT DETECTED"

        _,buffer=cv2.imencode('.jpg',frame)

        frame_bytes=buffer.tobytes()

        yield(
            b'--frame\r\n'
            b'Content-Type:image/jpeg\r\n\r\n'
            +frame_bytes+
            b'\r\n'
        )

@app.route('/')
def index():
```

```
    return render_template_string(
        html_page,
        target=TARGET_SHAPE,
        status=last_status
    )

@app.route('/set/<shape>')
def set_shape(shape):

    global TARGET_SHAPE
    global smooth_x
    global smooth_y

    if shape in [
        "CIRCLE",
        "SQUARE",
        "TRIANGLE"
    ]:

        TARGET_SHAPE=shape
        smooth_x=None
        smooth_y=None

    return redirect('/')

@app.route('/video_feed')
def video_feed():

    return Response(
        generate_frames(),
        mimetype='multipart/x-mixed-replace; boundary=frame'
    )

if __name__=="__main__":

    try:

        app.run(
            host="0.0.0.0",
            port=5000,
            debug=False
        )

    finally:
        try:
            requests.get(
                f"http://{camera_ip}/control?var=led_intensity&val=0"
            )

            print("LED isključen")

        except:
            pass
```

P.2 – Programski kod za ESP32 DevKit

U ovom prilogu navodi se izvorni programski kod za ESP32 DevKit mikroupravljač. Kod prima koordinate putem serijske komunikacije, provjerava njihovu ispravnost, preslikava ih u kutove aktuatora i upravlja pomicanjem po dvije osi.

```
#include <ESP32Servo.h>

Servo servoX;
Servo servoY;

int servoXPin = 18;
int servoYPin = 19;

int angleX = 90;
int angleY = 90;

void setup() {
  Serial.begin(115200);

  servoX.attach(servoXPin);
  servoY.attach(servoYPin);

  servoX.write(90);
  servoY.write(90);

  Serial.println("ESP32 2-servo ready");
}

void loop() {
  if (Serial.available()) {
    String data = Serial.readStringUntil('\n');

    int commaIndex = data.indexOf(',');
    if (commaIndex > 0) {
      int x = data.substring(0, commaIndex).toInt();
      int y = data.substring(commaIndex + 1).toInt();

      if (x >= 0 && x <= 640 && y >= 0 && y <= 480) {

        angleX = map(x, 0, 640, 150, 30);
        // Y smjer
        angleY = map(y, 0, 480, 30, 150);

        angleX = constrain(angleX, 30, 150);
        angleY = constrain(angleY, 30, 150);
        servoX.write(angleX);
        servoY.write(angleY);

        Serial.print("X=");
        Serial.print(x);
        Serial.print(" Y=");
        Serial.print(y);
        Serial.print(" AngleX=");
        Serial.print(angleX);
        Serial.print(" AngleY=");
        Serial.println(angleY);
      }
    }
```

```
    }
}
```

P.3 – Fotografije izvedbe projekta

U ovom prilogu prikazuju se dodatne fotografije izrađenog sustava. Fotografije prikazuju povezivanje elektroničkih komponenti, položaj kamere, aktuatorski mehanizam i izgled sustava tijekom rada.

